

Software fault prediction using data mining, machine learning and deep learning techniques: A systematic literature review[☆]

Iqra Batool, Tamim Ahmed Khan^{*}

Department of Software Engineering, Bahria University, Islamabad, Pakistan

ARTICLE INFO

Keywords:

Software fault prediction
Defect prediction
Machine learning techniques
Data mining techniques
Deep learning techniques
Performance measures

ABSTRACT

Software fault/defect prediction assists software developers to identify faulty constructs, such as modules or classes, early in the software development life cycle. There are data mining, machine learning, and deep learning techniques used for software fault prediction. We perform analysis of previously published reviews, surveys, and related studies to distill a list of questions. These questions were either answered in the past but needed a fresh look or they were not considered at all. We justify why answers to newly added questions are important and divide previous work based on data mining, machine learning, and deep learning and compare their performance. We study which datasets were commonly used and what comparison criteria were mostly adopted for software fault prediction. We select 68 primary studies from a wide list of initially selected set following our quality assessment criteria and present answers to our research questions.

1. Introduction

A fault in a software system is described as a structural flaw that could cause the system to fail in the future [1]. Software fault prediction is an important and necessary activity to increase software quality and minimize maintenance effort during the initial phases of software development. Early prediction of faults can lead to timely resolution of these faults and delivery of maintainable software as well. Software fault/defect prediction helps improve resulting software quality, where faults are predicted based on previous knowledge in the form of datasets [2]. There are Data Mining (DM), Machine Learning (ML), and Deep Learning (DL) techniques that we use for fault prediction. These techniques are used to build models that can be used for predicting faulty and non-faulty classes.

Software Fault/defect Prediction (SFP) is accomplished by dividing modules/classes into fault-prone and non-faulty categories [3]. We make use of fault datasets from similar projects or previous updates of the software systems. We then apply developed models to current software projects' modules, classes or methods, and classify them as fault-prone or non-faulty/fault-free modules, classes, or methods in a system under test (SUT). We enable verification experts to concentrate their time and resources on the problematic areas of the system under development using software fault/defect prediction [4]. SFP is highly beneficial when we have limited time and financial resources and time remaining before release only allows us to test part of the system. In this case, we can benefit from our fault prediction methods and channel our efforts and resources to places where we predict the presence of faults. Another benefit of SFP is in the shape of fault distribution awareness in advance where we make use of previous data to improve the quality of the upcoming releases since we have an idea about the presence of faults in a particular part of the SUT before testing is underway. However, there are challenges associated with software fault since we lack techniques that can be

[☆] This paper is for regular issues of CAEE. Reviews were processed by Guest Editor Dr. G. Botella and recommended for publication.

^{*} Corresponding author.

E-mail addresses: iqra@bahria.edu.pk (I. Batool), tamim@bahria.edu.pk (T.A. Khan).

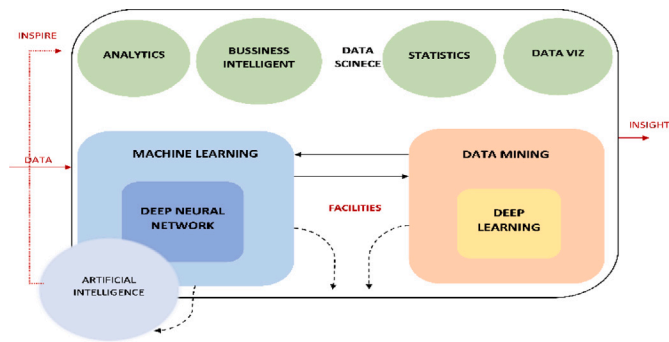


Fig. 1. Evaluation of Techniques.

applied. We do not have SFP implementations that can be instantly applied as predictors to assist development and testing for an early prediction. We also lack software faults data of previous applications in the shape of datasets that we can use to extensively investigate the usefulness of SFP models and improve them constantly. We also require metrics collection and sharing repositories that have specific or related software-related data [4].

We find systematic literature review (SLR) on fault prediction, e.g., [3,5]. We find a few limitations with existing work. Firstly, there are surveys that provide a good insight into fault prediction methods, yet we find them considering research conducted four years ago. Examples of such surveys and reviews include [6,7]. This makes the basis of our first research question which is addressed by existing research but includes work up to the year 2017.

Secondly, we find most of the reviews consider data mining or machine learning techniques since deep learning for fault prediction are quite a new consideration for researchers. The relation between DM, ML, and DL is already well-known as shown in Fig. 5. It is for this reason that we provide a taxonomy of DM, ML, and DL techniques providing a comprehensive account of their usefulness. Thirdly, we find a gap in the shape of the non-availability of a comprehensive and exhaustive list of datasets that researchers can consider for training their models. We show the involvement of DM, ML, and DL techniques as well as the importance of datasets in Fig. 1. We, therefore, compile the most recent list of datasets that are used for fault prediction purposes for researchers to have an updated account, considering both public and private datasets. Lastly, we find a variety of considerations while researchers compare their results and conduct validity and reliability checks of their models. We require how many different ways researchers have compared their results and how they have made use of concepts such as sensitivity analysis, specificity, precision, recall, F1-score, cross-validation, etc. We include all these considerations in our list of research questions and we select papers in the last decade i.e., (2010–2021). We consider a couple of studies outside our last decade e.g. [8,9], due to their citation score and topic relevance.

We contribute by providing an extensive review of DM, ML, and DL techniques used for software fault prediction. We identify key categories of these fault prediction techniques. We also identify how these techniques are effective to software fault prediction and list shortcomings of existing techniques.

We provide research motivation in Section 2, and research methodology in Section 3. We present our key findings in Section 4. We, finally, outline future directions and present conclusion in Section 5.

2. Literature review and motivation

Researchers conducted literature reviews, mapping studies, and surveys considering data mining, machine learning, and deep learning techniques for software fault/defect prediction. We give a critical account of these techniques while justifying our gap, research questions, and objectives. Systematic literature survey presented in [8] report machine learning as a popular technique for software fault prediction. In their study, they used NASA dataset located in the PROMISE Repository for SFP. A survey similar in design to [8] but more extensive in terms of included number of studies and analyses is published in [10]. The primary purpose of this study is the evaluation of context, independent variables, and modeling techniques. A quantitative model is developed considering 19 studies for comparing metrics output such as F-measure, precision, and recall. This study reports that object-oriented (OO) metrics perform better than complexity metrics and models that use a combined range of metrics outperform the use of process metrics. We find the use of software change metrics in [11] to predict faulty classes using machine learning techniques. The open-source WEKA tool is used for implementing ML models and open-source software Keel (<http://www.keel.es>) is used for the development of a hybrid model. The data is taken from the GIT repository. Experiments are conducted on numerous versions of the Eclipse project and change metrics are extracted from the GIT repositories. The aim and scope of our review are different from the above studies.

K Sorensen clustering is used for the development of the model in [12], which calculates clustering distance using Sorensen distance to predict software faults. Datasets for this review were extracted from the online NASA MDP PROMISE Repository. MATLAB is used for the implementation of the proposed model. The author combined the requirement and code metrics to build a model known as the Alliance metric model. The results indicate that the model developed using the K-Canberra Mean algorithm is more accurate and provides better fault prediction.

A systematic review of existing software fault prediction and software fault tolerance papers is presented in [13]. This review focused on statistical and machine learning approaches for fault prediction. It also utilized fault tolerance techniques like n-version and n-check programming. The capability of Ensemble Learning Algorithm(ELA) with eight NASA software defect datasets in Fault Prediction (FP) software modules is presented in [14]. Authors evaluated and compared its output with Feature selection(FS) and both with FS and Data Balancing (DB) techniques. The motivation behind this was to prune non-relevant features and balance classes; and then learn an ELA that performs better than from learning on the whole features and in balanced classes. An investigation of software ensemble methods is presented in [15]. They evaluated the performance of seven ensemble learning methods and concluded that Random Forest with j48 as a base learner achieved the highest precision, recall, and G-mean values.

Both feature selection and instance reduction are presented in [16] to improve the quality of software datasets used by classification models for software fault prediction. This study analyzed different selection algorithms for fault prediction. NASA and Eclipse datasets are used for this empirical research. Different feature extraction techniques such as Principal Component Analysis(PCA), Linear Discriminant Analysis (LDA), Kernel-based Principal Component Analysis (K-PCA), and Autoencoders with SVM classifiers are explored [17]. The paper used nine open-source software-systems written in Java from PROMISE Repository and found SVM as the best autoencoder classifier. The paper used Spyder for feature and IBM SPSS for hypothesis testing using the Friedman test. Different ML algorithms that are used mainly for test-case generation, refinement, and evaluation are studied in [18]. This study considers 48 primary studies that are categorized according to study type, testing activity, and ML algorithm employed to automate the testing activity.

SLR presented in [19] study use of traditional machine learning, deep learning, and hybrid learning for SFP. The authors consider performance outcome in detail, especially in cross-project and just-in-time. A primary study on 40 most relevant Clarivate Analytics indexed Journal papers is presented in [1]. The authors reported NASA datasets to be widely used ones and they found McCabe as an effective metric. They also evaluated which of the performance metrics were more commonly used. A survey on deep learning techniques is presented in [20]. In this study, the authors discussed various deep learning techniques which are used for software defect prediction. They conclude that Long Short Term Memory(LSTM), Convolutional Neural Network(CNN), Stacked Autoencoder and Deep Belief Network (DBN) are the most common techniques of deep learning used for software fault prediction.

We consider studies for a possible inclusion for an up-to-date and more comprehensive analysis and we conduct a quality assessment step. Deep learning algorithms and their applications for fault prediction is a newer topic addressed recently and we make it part of our research questions providing a critical overview in our study. We consider machine learning, deep learning, and hybrid learning approaches that significantly improve prediction performance, aiming to predict defects of both cross-project and within-project. We additionally provide the most recent analysis with a bifurcation of their usage in data mining, machine learning and deep learning, as well. We study what type of empirical validation and comparison measures are used. We highlight the use of dataset pre-processing by applying instance filtering along with attribute selection to achieve better results. We present our research methodology in the following section.

3. Research methodology

We employed systematic literature review (SLR) technique to identify DM, ML, DL techniques used for software fault prediction. We make use of SLR technique presented in [21]. We divide our process into three phases, “Planning the review”, “Conducting the Review”, and “reporting the review” following guidelines presented in [21]. We present their details in Fig. 2.

3.1. Phase 1: Planning the review

We perform the following steps while planning this review:

- Designing the research questions
- Choosing research repositories
- Creating search string for extraction of articles
- Defining article’s inclusion and exclusion criteria
- Defining quality assessment criteria

3.1.1. Research questions

This systematic literature review focuses on DM, ML, and DL methods used for software fault prediction. A sequence of Research Questions (RQs) is formulated and defined based on our target objectives. We elaborate our research questions and motivation behind these research questions in Table 1.

3.1.2. Data repositories

We select search repositories based on previous experience and recommendations in [22]. We consider digital libraries of IEEE Xplore, Science Direct, Springer Link, Wiley and Google Scholar. Since the search mechanism is different in all data repositories, we, therefore, tailor search queries accordingly.

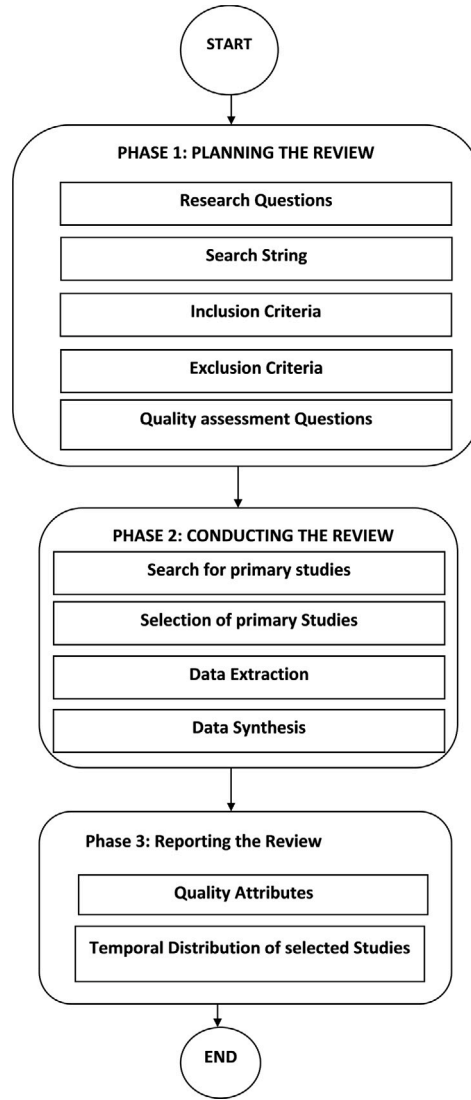


Fig. 2. Systematic Review Process for Software Fault prediction..

3.1.3. Search strings

We adopt a focused search process in a manner that our research questions dominated the whole process. We choose keywords and alternatives based on available literature in the context of software fault prediction. We categorize our research into two groups; the first group includes software fault prediction and the second group comprises terms related to software fault prediction methods. Finally, we apply the content analysis technique to obtain category frequencies. We explain our search strings in [Table 2](#).

3.1.4. Quality assessment of selected studies

We determine the quality of chosen studies considering our inclusion and exclusion criteria given below:

3.1.5. Inclusion criteria

The inclusion and exclusion criteria of our study are based on the principles given by [21]. Following are the inclusion criteria for the selection of primary studies.

- IC 1: The selected study should be a conference paper, a journal paper, or a book chapter.
- IC 2: The selected study should focus on software fault prediction using DM /ML/DL techniques
- IC 3: The selected studies should discuss the challenges and performance of fault prediction techniques.
- IC 4: The selected studies should present fault prediction models or techniques.
- IC 5: The selected studies should be available in full-text articles; the preferred language should be English.
- IC 6: The selected studies should be published between 2010–2021.

Table 1
Research questions.

Q No.	Research questions	Motivation
RQ 1	Which DM, ML, DL techniques are used for SFP?	Identify DM, ML, and DL techniques commonly being used in SFP.
RQ 2	Which dataset repositories for acquiring dataset?	List repositories and their quality assessment concerning nature of repository, dataset size, and use of dataset for DM, ML, DL techniques
RQ 2.1	Are there models and techniques that are used on the same data sets?	Investigate different models/techniques used with same datasets
RQ 2.2	Which studies use which datasets?	Investigate dataset values based on the gold standard, silver standard, and bronze standards
RQ 2.3	are there the same models applied to combination of different data sets?	Investigate the same models used with a combination of different datasets
RQ 2.4	Are there different models applied on a combination of different datasets	Investigate different models used with different dataset
RQ 3	Which are the most used metrics and associated datasets used for SFP?	Investigate the most used metrics and their reasons why they were adopted
RQ 4	Which performance measures are adopted and which tools for model evaluation are used?	Assess performance measures used and tools support discussed for DM, ML, and DL techniques
RQ 5	How models are prepared for software Fault prediction?	Investigate performance of other models.

Table 2
Search string.

Group	Search string
software Fault Prediction	Fault prediction or Fault detection, Defect Prediction AND
Software Fault Prediction using DM, ML, DL technique	Software fault prediction using Data mining techniques. Software fault prediction using Machine Learning techniques. Software Fault Prediction using Deep learning techniques

3.1.6. Exclusion criteria

We exclude primary studies based on the following exclusion criteria.

- EC 1: The studies that do not answer the question defined in Section 1.
- EC 2: The studies that do not discuss Software fault prediction in DM, ML, and DL context.
- EC 3: The studies that do not highlight challenges or methods used for fault prediction.
- EC 4: The studies that are not written in English.
- EC 5: The studies that are not reaching the acceptance threshold considering our quality assessment criteria defined in Table 3.

Once we do an independent evaluation of selected studies, we award scores by each researcher in the group using the following scoring method.

- We award a score of '1' to papers that present the correct answer to the checklist question.
- We award a score of '0.5' to papers that present a partial answer to checklist question.
- We award a '0' score in case a paper does not answer desired checklist questions at all.

We award a quality score to our selected studies. It is pertinent to highlight that a selected paper can score a maximum of '10' points since we award a score of '1' point in case the paper is fully satisfying our posed quality assessment criteria. Our quality assessment questions include certain categories of questions. The first three questions are related to quality of the article whereas the fourth question is aligned with one of our posed research questions to explicitly find out which of the DM, ML, or DL techniques separately or in conjunction are used in the study. Our fifth and sixth questions are related to datasets, their quality, their size and their origin whereas seventh question is to see if the selected study presents a comparison of results. We finally address the quality of results, use of performance measures, and threats of validity of results in the last two questions. We present our questions in as shown in Table 3 where we also show what possible marks can be provided since, in case of Q5 and Q6, we can only award '1' or '0' marks.

Table 3
Quality assessment questions.

Q No.	Quality question	yes	No	Partly
Q1	Are the aim of the research defined explicitly?	1	0	0.5
Q2	Is there experimental design appropriate and justifiable?	1	0	0.5
Q3	Is the research methodology repeatable?	1	0	0.5
Q4	Are the DM, ML or DL techniques clearly justified and defined?	1	0	0.5
Q5	Is the dataset size adequate with respect to size, origin and quality?	1	0	–
Q6	Is the dataset available publicly or privately?	1	0	–
Q7	Are there any comparative analysis between chosen DM, ML or DL techniques conducted	1	0	–
Q8	Are results and findings are clearly stated?	1	0	0.5
Q9	Are performance measures used to assess the SFP models clearly defined?	1	0	0.5
Q10	Are the limitation of study specified?	1	0	0.5

Table 4
Selected studies.

E-Database	Paper extracted through search terms	Inclusion/exclusion based on title and abstract	Inclusion/exclusion based on introduction and conclusion	Inclusion/exclusion based on full text	Total selected articles for primary study	Percentage of the final selected articles (N=68)
Google Scholar	709	82	38	15	16	23.52%
IEEE Xplore	586	51	40	25	26	38.23%
Elsevier	304	20	13	12	13	19.11%
Springer Link	169	19	13	10	10	14.70%
Wiley Online Library	25	8	6	3	3	4.61%
Total	1793	187	110	68	68	100%

3.2. Phase 2: Conducting the review

We select papers following our search process and shortlist studies following inclusion and exclusion criteria. We, finally, process initially selected studies through our quality assessment procedure. We arrive at a list of shortlisted studies. We discuss details of our process and the associated activities in selection process in the forthcoming sections.

3.2.1. Primary study selection

We identify a significant number of reports during the selection of studies process and we use the Tollgate approach by Afzal and Torkar [23] to optimize selection process. Tollgate approach consists of five phases as shown in Fig. 3 and described in Table 3. We select, in the first stage, 1793 articles based on keywords and research items from all databases. We choose 187 articles based on title and abstract inclusion and exclusion criteria in the second phase. We include those articles whose introduction and conclusion include answers to our proposed questions in the next step and we find 110 as the resulting articles. We apply duplication removal and we are left with 68 articles applying these steps as shown in Fig. 3. We find 38.23% of selected studies from IEEE, 23.52% from Google Scholar, 19.11% from Elsevier, 14.70% from Springer Link (14.70), and 4.61% from Wiley databases.

3.2.2. Data extraction

We extract data for formulating answers to our research questions. We identify characteristics such as research methodologies, study type, and limitations connected with these studies. We extract factors from our selected studies that we use to analyze our selected studies. We use these factors to further analyze and investigate our selected studies and we present a discussion and interpretation of these findings before we present our results of this investigation in Table 6. We provide results and our analysis of results in this section. Our explanation of constructs used in Table 6 is as follows:

1. Software fault prediction: We consider Software Fault prediction as an evaluation factor because it is our primary concern in this study.
2. Software defect prediction: As listed in our search string Table 2 this is the term which we use in our study of software fault.
3. Use of DM, ML, and DL Techniques: We use Data mining, machine learning, and Deep learning techniques as evaluation factor because it maps on to our research question RQ1, RQ2, RQ3, RQ4, and RQ5.
4. Tools Usage: We intend to highlight which of the software tools are in popular use in our primary studies.
5. Perform Measure: We use this factor in our study to investigate different performance measures used for DM, ML, and DL techniques and with what frequency.
6. Use of Metrics: We intend to map which of the metrics and which of the datasets based on these metrics are widely used by researchers in our primary studies.

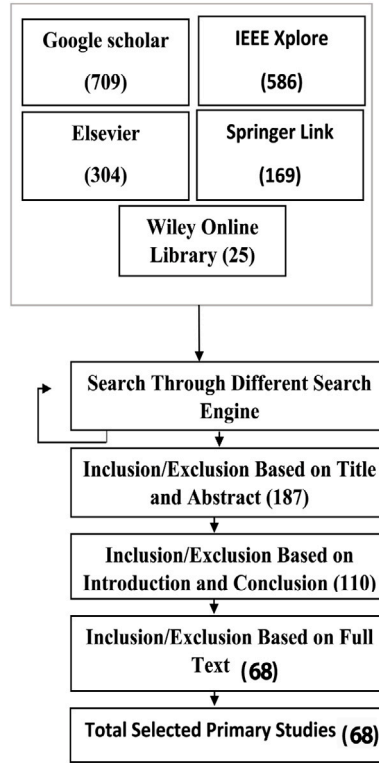


Fig. 3. Article selection process.

7. **Datasets:** We investigate used datasets based on their origin, their size, and their quality. We also intend to find out which dataset is used in which percentage.
8. **Feature Selection:** Feature subset selection is an important area of concern and we report how many studies consider this aspect.

3.2.3. Data synthesis

We organize information gathered from primary studies and conduct a filtering processing considering our inclusion/exclusion criteria as well as our research questions in this stage. We present our quality assessment results in [Table 5](#).

3.3. Phase 3: Reporting the review

We conduct an assessment process and both of the researchers do an independent analysis of selected primary studies. It is pertinent to mention that these studies are distilled by researchers independently and then a collective result through a series of meetings is generated.

We find a list of selected studies for reporting final results at the end of this process. We also address the temporal distribution of the studies in order to define a resulting pattern, study types, and many other research outputs that we discuss in the results and discussion section. The subsequent subsections go through the phases in greater depth.

3.3.1. Quality attributes

We evaluate all of the selected studies for quality considering our quality evaluation criteria so that may be included in our final collection. We do it in two phases. We give scores independently in the first phase and then we do a committee stage where each article is discussed for final inclusion in the list of selected studies. We present the result of our analysis in [Table 5](#) in shape of the quality score for each primary study. This is based on our formulated quality questions shared in [Table 3](#).

3.3.2. Temporal distribution of selected studies

We select a total of 68 selected studies where 50 percent of research papers are published between 2010 and 2015, and the same percentage of publications (50 percent) are published between (2015–2021). We find noticeable work carried out in the context of software fault prediction. We used snowballing technique to extract specific work from citations of initially selected studies. We

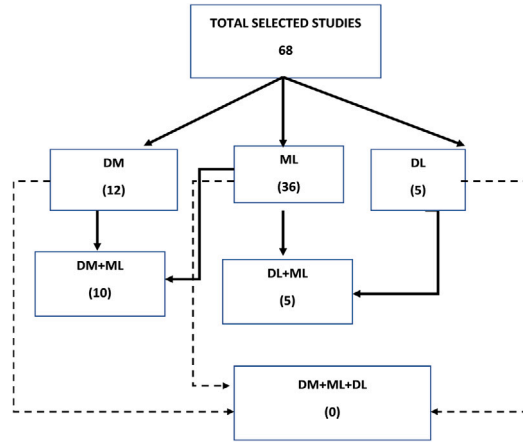


Fig. 4. Classification of techniques.

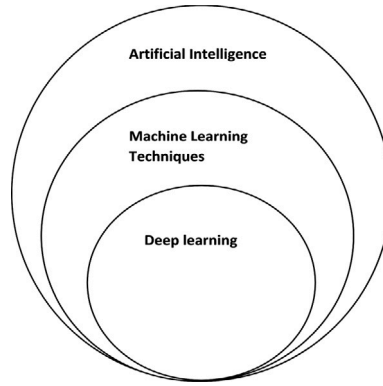


Fig. 5. Relation between Data Mining, Machine Learning, and Deep Learning [86].

present our selection process using Fig. 3 and list of selected studies in Table 5. We provide a division of selected studies according to indexing databases and journals in Table 4.

We present the distribution of studies within our selected studies applying inclusion and exclusion criteria in Fig. 4. It is pertinent to note that we also show, using Fig. 4, how many of the selected studies involve the use of DM, ML, DL, DM+ML, ML+DL, DM+DL, and DM+ML+DL techniques. It is important to highlight that we did not find any study that involves the use of DM+DL and the use of DM+ML+DL techniques. We present, in Fig. 5, where we see the relation between these techniques.

4. Results and discussion

We consider factors explained in Section 3.2.2 and apply the data extraction step to our primary studies considering these factors. We do an impact analysis of each study considering these factors and formulate our analysis of results. We provide our analysis of published sources in Table 4, which summarizes details of publications, publishing journals, or conferences along with percentages.

We present, in Fig. 6, the distribution of studies from the year 2010 to 2021. We notice that there is a shift of methodology from data mining to machine learning in the first half of the outgoing decade and the last two years show a shift of focus to the use of deep learning techniques.

4.1. RQ1: Use of data mining, machine learning and deep learning techniques for SFP

We find a variety of techniques from a wide variety of data mining techniques used for fault prediction in our selected primary studies. We provide a taxonomy of these techniques considering DM, ML, and DL techniques in turn.

Table 5

Quality score of selected Primary Studies(PS).

SS	Study Ref	Type	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8	Q9	Q10	Score
SS1	[24] Gao et al.	DM+ML	1	1	1	1	1	1	1	1	0.5	0.5	9
SS2	[25] zheng et al.	ML	1	0.5	1	1	1	1	1	1	0.5	1	9.5
SS3	[26] Li and Zhang et al.	ML	1	0.5	0.5	1	0	1	1	0.5	1	1	8.5
SS4	[27] Sun et al.	ML	1	1	0	1	0	1	1	1	1	0.5	8
SS5	[28] Arar et al..	ML+DL	1	1	1	1	1	1	0	1	1	1	9
SS6	[29] wong et al.	ML	1	0.5	0	1	1	1	1	1	1	1	9.5
SS7	[30] Malhotra et al.	DM+ML	1	1	1	1	1	1	1	1	0.5	1	9
SS8	[31] Zibula et al.	DM	1	1	1	1	1	1	1	1	1	1	9.5
SS9	[32] Lv et al.	DL	1	1	1	1	0	1	1	1	1	1	9.5
SS10	[33] cai et al.	ML	1	1	1	1	1	1	1	1	0.5	1	9.5
SS11	[34] Turhan and X et al.	ML	1	1	1	1	1	1	0	1	1	1	9
SS12	[35] Catal et al.	DM+ML	1	1	1	1	0	1	1	1	0.5	1	8
SS13	[16] Liu et al.	ML	1	1	1	1	1	1	0	0	1	1	8
SS14	[36] Moeyersoms et al.	DM	1	1	1	1	1	1	1	1	1	1	8.5
SS15	[7] Manjula et al.	DL	1	1	1	1	1	1	0.5	0.5	0.5	1	7.5
SS16	[37] Jin et al.	DM+ML	1	1	1	1	1	1	1	0	0.5	0.5	8
SS17	[38] Deng at al.	ML	1	1	1	1	1	1	0.5	0.5	0.5	1	8.5
SS18	[39] Sing and pal.	DM	1	1	1	1	1	1	1	1	1	0	9
SS19	[40] wang and liu.	DL	1	1	1	1	1	1	0	0	1	1	8
SS20	[41] Malhotra et al.	ML	1	1	1	1	1	1	1	1	1	0.5	9.5
SS21	[42] Malhotra et al..	ML	1	1	0	1	1	1	1	0.5	1	0.5	8.5
SS22	[11] Choudhary et al.	ML	1	1	0	1	1	1	1	1	1	1	9
SS23	[2] Turabieh et al.	DM+ML	1	1	0.5	1	1	1	1	1	1	1	9.5
SS24	[43] Huda et al.	ML	1	1	1	0.5	1	1	1	1	1	1	9.5
SS25	[44] Abebe et al.	DM+ML	1	1	0.5	1	1	1	0	0	1	1	7.5
SS26	[45] Malhotra et al.	ML	1	1	1	1	1	1	0.5	0.5	1	1	9
SS27	[46] Zhengand et al.	DL	1	1	1	1	1	1	0	0	1	1	8.5
SS28	[47] Sing et al.	ML	1	1	1	1	1	1	1	1	1	1	10
SS29	[48] Ren et al.	ML	1	1	1	1	1	1	0.5	0.5	1	1	9
SS30	[49] Jayanthi et al.	ML	1	1	0.5	1	1	1	0	0	1	1	7.5
SS31	[50] Jin et al.	DM+ML	1	1	0	1	1	1	0.5	0.5	1	1	8
SS32	[51] Fan et al.	DM	1	1	1	1	1	1	0	0	1	1	8
SS33	[22] Chen et al.	DM	1	1	1	1	1	1	0	0	1	1	8
SS34	[52] Chen et al.	ML	1	1	1	1	0	1	1	0	1	1	8
SS35	[53] Park and Hong.	DM	1	1	1	1	1	1	1	1	1	0.5	9.5
SS36	[54] Hall et al.	ML	1	1	1	1	1	1	0	0	1	0.5	7.5
SS37	[55] Ronge et al.	ML	1	1	1	1	1	1	1	1	1	0.5	9.5
SS38	[56] Akour et al.	ML	1	1	1	1	1	1	1	1	1	1	10
SS39	[57] Singhad bhatia et.	ML	1	1	1	1	1	1	1	1	1	1	10
SS40	[58] Zhou et al.	ML	1	1	1	1	1	1	1	0.5	0.5	1	9
SS41	[59] Abdi et al.	ML	1	1	1	1	1	1	0	1	1	1	9
SS42	[60] Rathore et al.	ML	1	1	0	1	1	1	0	1	0.5	1	8.5
SS43	[61] Phan dand et al.	ML	1	1	1	1	1	1	1	1	1	1	10
SS44	[62] Arshad et al.	DM	1	0.5	1	1	1	1	0	1	1	1	9
SS45	[63] Cao et al.	DM	1	1	1	0	1	1	1	0.5	1	1	8.5
SS46	[64] Shanthini et al.	ML	1	1	1	1	1	1	0	0	1	1	7.5
SS47	[65] Sharma et al.	ML	1	1	1	1	1	1	0	0	1	1	7
SS48	[66] Rana et al.	ML	1	1	1	1	1	1	0	0	1	1	7
SS49	[67] Coelho, R. A.	ML	1	1	1	1	1	1	0	0	1	1	7.5
SS50	[68] Sethi et al.	DM	1	1	1	1	0	0	0	0	1	1	6
SS51	[69] Chatterjee et al.	DM	1	1	0	1	1	1	0	0	1	1	7.5
SS52	[70] Kaur et al.	DM	1	1	1	1	1	1	0	1	0.5	1	8.5
SS53	[71] Wu and F. et al.	ML	1	1	1	1	1	1	0.5	0	0.5	1	8.5
SS54	[72] Bejjanki et al.	ML	1	0.5	1	1	1	1	1	0.5	1	1	8.5
SS55	[73] Khuat et al.	ML	1	1	1	1	1	1	0.5	1	1	1	9.5
SS56	[74] Chen et al.	ML+DL	1	1	1	1	1	1	1	0	1	1	9
SS57	[75] sheppred et al.	ML	1	1	1	1	1	0	1	1	0.5	1	8.5
SS58	[76] Weis et al.	DM+ML	1	1	0.5	1	1	1	0.5	1	0.5	1	8.5
SS59	[77] Munadada et al.	ML+DL	1	1	1	1	1	1	1	1	1	1	9.5
SS60	[78] Zhang et al.	DM+ML	1	1	1	1	1	1	1	1	1	1	9
SS61	[79] Al qasem et al.	ML+DL	1	1	0	1	1	1	1	0.5	1	1	9.5
SS62	[80] Rheman et al.	ML	1	0.5	1	1	1	1	0.5	0.5	1	1	8.5
SS63	[81] Al Qasem et al.	ML+DL	1	1	1	1	0	1	1	1	0.5	1.5	8
SS64	[13] Arora et al.	DM	1	1	0	1	1	1	1	1	1	1	9.5
SS65	[82] Jinyin Chen et al.	ML+DL	1	1	1	1	1	1	1	1	1	1	10

(continued on next page)

Table 5 (continued).

SS	Study Ref	Type	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8	Q9	Q10	Score
SS66	[83] Jinyin Chen et	DL	1	1	1	1	1	1	1	0.5	1	1	9.5
SS67	[84] Elife Öztürk et al.	ML	1	1	1	0.5	1	1	1	1	1	1	9.5
SS68	[85] Balaram et al.	ML	1	1	1	0.5	1	1	1	1	1	1	9.5

Table 6

Constructs table.

Sr.NO	Constructs	Primary Studies	Frequency	Percentage
C1	Software fault prediction	[SS6], [SS7], [SS9], [SS12], [SS13], [SS14], [SS16], [SS17], [SS18], [SS20], [SS21], [SS22], [SS23], [SS25], [SS26], [SS27], [SS30], [SS34], [SS35], [SS36], [SS38], [SS39], [SS41], [SS42], [SS44], [SS46], [SS47], [SS49], [SS50], [SS52], [SS53], [SS54], [SS56], [SS57], [SS58], [SS60], [SS61], [SS62], [SS43], [SS64], [SS65], [SS68]	42	61.76
C2	Software defect prediction	[SS1], [SS2], [SS3], [SS4], [SS5], [SS8], [SS10], [SS11], [SS15], [SS19], [SS28], [SS29], [SS30], [SS32], [SS33], [SS37], [SS40], [SS43], [SS45], [SS48], [SS51], [SS59], [SS66], [SS67]	25	36.76
C3	Data mining techniques	[SS8], [SS14], [SS18], [SS33], [SS35], [SS44], [SS50], [SS52], [SS52], [SS58], [SS63]	11	16.17
C4	Machine learning techniques	[SS1], [SS2], [SS3], [SS4], [SS6], [SS7], [SS8], [SS10], [SS11], [SS12], [SS13], [SS17], [SS20], [SS21], [SS22], [SS23], [SS24], [SS25], [SS34], [SS36], [SS28], [SS29], [SS30], [SS34], [SS36], [SS37], [SS38], [SS39], [SS41], [SS42], [SS46], [SS47], [SS48], [SS50], [SS54], [SS55], [SS56], [SS57], [SS59], [SS68]	40	58.82
C5	Deep learning techniques	[SS9], [SS15], [SS19], [SS27], [SS62], [SS64], [SS66], [SS67]	8	11.76
C6	Other Models	[SS5], [SS16], [SS26], [SS30], [SS31], [SS32], [SS40], [SS43], [SS45], [SS49], [SS51], [SS60], [SS61], [SS65]	14	20.58
C7	Tools usage.	[PS1], [SS3], [SS7], [SS12], [SS15], [SS23], [SS27], [SS30], [SS35], [SS41], [SS53], [SS62], [SS64], [SS65]	14	20.58
C8	Performance measure	[SS1], [SS2], [SS3], [SS4], [SS5], [SS7], [SS8], [SS9], [SS10], [SS14], [SS15], [SS16], [SS23], [SS24], [SS25], [SS26], [SS29], [SS30], [SS40], [SS41], [SS44], [SS46], [SS50], [SS53], [SS55], [SS57], [SS58], [SS62], [SS63], [SS64], [SS65], [SS66], [SS67], [SS68]	34	50
C9	Use of software metrics	[SS1], [SS3], [SS7], [SS8], [SS9], [SS11], [SS13], [SS17], [SS24], [SS28], [SS29], [SS31], [SS32], [SS33], [SS41], [SS44], [SS48], [SS52], [SS66], [SS67], [SS68]	21	30.88
C10	Datasets	[SS1], [SS2], [SS3], [SS4], [SS5], [SS6], [SS7], [SS8], [SS10], [SS11], [SS13], [SS14], [SS15], [SS16], [SS18], [SS21], [SS22], [SS23], [SS24], [SS25], [SS27], [SS26], [SS29], [SS30], [SS31], [SS32], [SS34], [SS37], [SS38], [SS39], [SS40], [SS41], [SS42], [SS43], [SS44], [SS45], [SS46], [SS50], [SS51], [SS54], [SS55], [SS57], [SS58], [SS59], [SS60], [SS61], [SS63], [SS63], [SS64], [SS66], [SS67], [SS68]	40	58.82

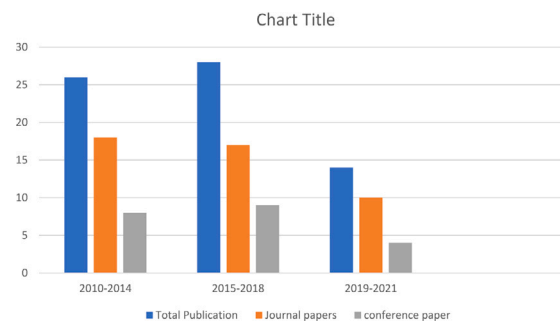


Fig. 6. Distribution of studies.

Table 7
DM studies with percentage.

Method	No. of studies (DM)	No. of studies (DM+ML)	Percentage
DT	2	1	4.41
NB	2	1	4.41
SVM	2	1	4.41
ARM	1	0	1.47
Fuzzy	1	1	2.94
J48	1	1	2.94
K star	1	1	2.94
K mean	1	1	2.94
C mean	1	1	2.94
Total Studies=12		Total Studies=10	32.34

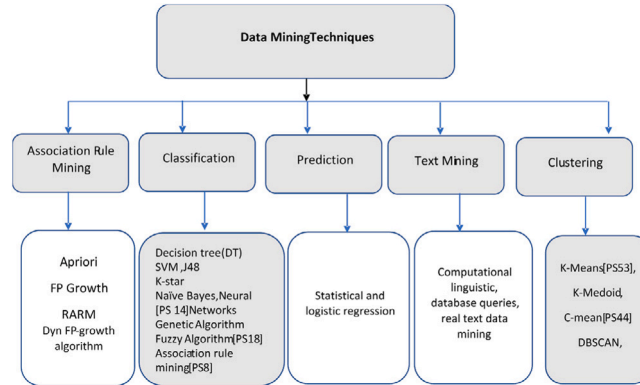


Fig. 7. Data Mining Techniques.

4.1.1. Use of data mining for SFP

We report use of data mining techniques in our selected studies that are presented in Table 5. These studies include [SS8], [SS14], [SS18], [SS33], [PS44], [SS53], [SS55], [SS58] and [SS63]. We find the following techniques as commonly used ones, considering our primary selected studies:

- Decision Trees (DT)
- Naïve Bayes(NB)
- Support vector Machine(SVM)
- Association Rule mining(ARM)
- fuzzy algorithms
- J48
- K star
- K means
- C means

We report most commonly used data mining techniques are as shown in Table 7.

If we consider data mining techniques classification given in [66], we highlight which ones are used in our selected primary studies, as shown in Fig. 7.

Next, we report those machine learning techniques that are used for software fault prediction in our selected primary studies. We report 36 studies out of our total of 68 studies that use machine learning techniques. These include [SS1], [SS2], [SS3], [SS4], [SS5], [SS7], [SS8], [SS9], [SS10], [SS11], [SS12], [SS13], [SS17], [SS18], [SS23], [SS24], [SS26], [SS28], [SS29], [SS30], [SS34], [SS36], [SS37], [SS38], [SS41], [SS42], [SS46], [SS47], [SS48], [SS50] and [SS59]. These machine learning techniques include:

- Decision Tree (DT)
- Naïve Bayes (NB)
- Ensemble Techniques (ET)
- Neural Networks (NN)
- Support Vector Machine (SVM)
- Rule Based Reasoning (RBR)
- Evolutionary Algorithms (EA)
- Miscellaneous (M)

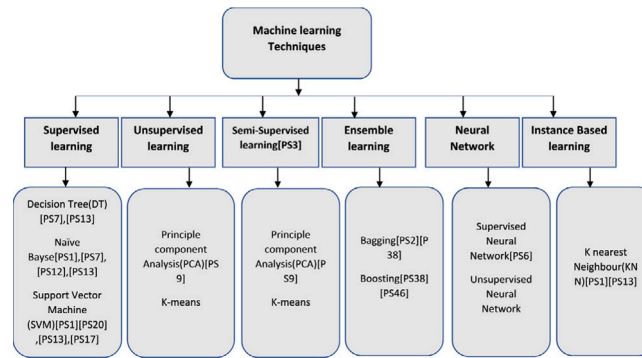


Fig. 8. Machine learning techniques.

Table 8

ML techniques with percentage.

Method	No. of studies(ML)	No. of studies(ML+DL)	Percentage
DT	6	1	10.29
BL	5	0	7.35
EL	2	1	4.41
NN	6	1	10.29
SVM	6	2	11.76
RBL	5	0	7.35
EA	2	0	2.94
Misc	3	0	4.41
Total studies =36		Total Studies=5	58.8

Our analysis shows a variety of data mining techniques used for software fault prediction such as association rule mining, clustering, classification neural network decision trees, Naïve Bayes, SVM, and Case-based reasoning. We find less work on the use of unsupervised or semi-supervised learning frameworks and the use of various datasets produce different results and performance outcomes.

There is only one study found that uses hybrid approaches [87] for fault prediction. We also find semi-supervised hybrid self-organizing map (HySOM) technique used for defect prediction which outperforms the use of unsupervised techniques as shown in [88]. However, AdaBoost and bagging also perform better in fault prediction. If we compare the use of data mining methods with the use of hybrid approaches, we find hybrid approaches produce better results as compared to the use of a single approach. This paves the way for more studies involving the use of hybrid approaches such as classification and regression Tree(CART).

4.1.2. Use of machine learning for SFP

If we consider machine learning techniques classification given in [89], we highlight which ones are used in our selected primary studies using Fig. 8.

We present machine learning techniques used for SFP in tabular format as shown in Table 8.

Machine learning models report good accuracy and the use of ensemble techniques such as bagging ensemble learning technique and Ada-boost is less frequent yet producing better results. We found more research tilt towards the use of academic origin public data sets and less towards private data sets. There is a clear need to use industrial data with more number of features. Another important aspect is use of method-level, class-level or module-level data in the datasets. Our analysis shows the use of class-level and method-level metrics in most of the studies as compared to module-level metrics with a need to do investigations using component or process-level data as well. Labeled datasets used for software fault prediction contain two classes with one showing presence and the other showing absence of faults. We find better results where we find use of techniques suitable to bi-class problems or where class-balance is introduced.

4.1.3. Use of deep learning for SFP

Finally, we report those studies that make use of deep learning technique. We find six studies i.e., [SS9], [SS15], [SS27], [SS52], [SS62],[SS64],[SS66],[SS67] that use deep learning techniques for software fault prediction. We find use of following deep learning techniques used in our selected studies.

- Convolutional Neural Network (CNN)
- Multi-layer Perceptron(MLP)
- Long Short Term Memory (LSTM)
- Deep Neural Network (DNN)

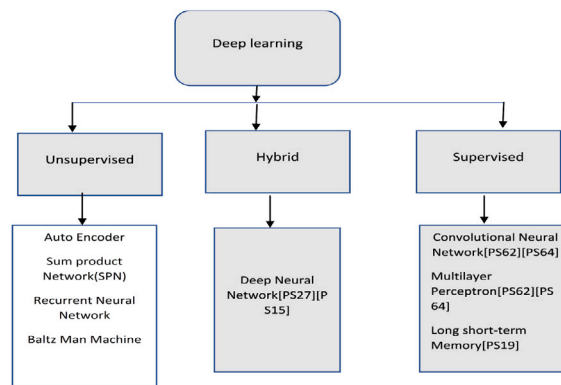


Fig. 9. DL Techniques.

Table 9
DL techniques with percentage.

Method	No. of studies	Percentage
CNN	2	2.94
MLP	1	1.47
LSTM	1	1.47
DNN	1	1.47
Total Studies= 5		7.35

We present deep learning techniques that we found in our selected studies in Fig. 9.

We present distribution of deep learning techniques in Table 9.

Deep learning techniques include supervised learning, unsupervised learning, reinforcement and hybrid learning. We find deep learning methods perform better as compared to simple Artificial Neural Networks (ANN). However, deep learning techniques' training time is generally higher than ANN. We also find use of back-propagation and over-fitting related issues in the use of deep learning. Back propagation can be used to calculate gradient function of each iteration [90]. Learning rate affects training time and we can reduce learning rate gradually for fine tuning of weights. Over fitting problems in deep neural network can be addressed using dropout which gives more better performance on supervised learning task. Batch normalization reduces co-variate shift, allowing deep neural networks to run faster. When the weights are updated during training, it normalizes inputs to a layer for each mini batch. Normalization stabilizes learning while shortening training epochs. Normalizing the output of the previous activation layer can improve the stability of a neural network. A model trained on one task is used on another related task in transfer learning. The knowledge gained while solving a specific problem can be transferred to another network that will be trained on a similar problem.

Deep learning frameworks aid in rapid modeling of a network without going into underlying algorithms. Each framework is designed for a specific purpose. TensorFlow is developed by Google brain which supports different languages such as Python, C++, R. It allows us to implement our deep learning models in both Central Processing Units (CPU) and Graphics Processing Unit (GPU). Keras is an API which is written in Python and runs on top of TensorFlow. It supports CNN and RNN and supports both CPU and GPU [91]. PyTorch can be used for developing deep neural networks as well as executing tensor computations [91]. PyTorch is a Python-based package that provides Tensor computations. PyTorch is a framework to create computational graphs. Caffe is developed by Yangqing Jia and it is open source. Caffe performance is better than other framework in processing speed and learning from images [91]. The Caffe Model Zoo framework allows us to easily access pre-trained models, allowing us to solve a variety of problems.

Deeplearnig4j is written in Java, which makes it more efficient than Python [91]. Deeplearning4j makes use of the ND4J tensor library, which allows it to work with multi-dimensional arrays or tensors. This framework is compatible with both CPUs and GPUs. Deeplearnig4j supports images, CSV, and plaintext. Deep learning models are implemented using large datasets. Different models have different implementation requirements. Some are implemented on CPU but most of them require GPU for their implementation. [81] implement deep learning techniques using 4 NASA dataset. They use hyper-parameter tuning but the one issue in their implementation is the dataset. Deep learning-based models can help in better fault localization [92]. We observe that different algorithms of deep learning performance are different on different datasets. Deep learning models can automatically extract features. In deep learning, we can use hybrid models for software fault prediction for better accuracy [93]. However, we do not find use of pseudo-labeling or multi-labeling and we do not see researchers using techniques such as transfer learning, etc.

4.2. RQ2: Repositories

We find a list of repositories that contain datasets that are used for empirical validation for predicting faults using the DM, ML and DL techniques. We find use of NASA Datasets in [SS2], [SS4], [SS5], [SS16], [SS24], [SS26], [SS29], [SS30], [SS31], [SS34], [SS38],

Table 10
Metrics used in PROMISE dataset.

Metrics Name	Known as
Weighted methods per class	WMC
Depth of Inheritance Tree	DIT
Number of Children	NOC
Coupling between object classes	CBO
Response for a Class	RFC
Lack of cohesion in methods	LCOM
Afferent couplings	Ac
Efferent couplings	Ec
Average McCabe's cyclomatic complexity	AVG-CC
Maximum McCabe's cyclomatic complexity	MAX-CC
M McCabe's cyclomatic complexity	CC
Average Method Complexity	AMC
Coupling Between Methods	CBM
Inheritance Coupling	IC
Cohesion Among Methods of Class	CAM
Measure of Functional Abstraction	MFA
Measure of Aggregation	MA
Data Access Metric	DAM
Lines of Code	LOC
Lack of cohesion in methods (by Henderson-Sellers)	LCOM3
Number of Public Methods	NPM
Number of files (compilation units)	NOCU

[SS39], [SS40], [SS44], [SS46], [SS50], [SS53], [SS54], [SS58], [SS59], [SS63] and [SS64]. In these studies, different techniques are applied on same datasets.

Some studies make use of the PROMISE repository for accessing datasets for SFP. They include [SS3], [SS15], [SS23], [SS26], [SS39], [SS40], [SS41], [SS55], [SS63], and [SS66]. We also find a subset of selected studies that make use of datasets available either on the web as public datasets such as Android repository or they are privately available and used in these selected studies. We find a number of studies that make use of private datasets and they include [SS7], [SS11], [SS13], [SS14], [SS22], [SS26], [SS29], [SS34], [SS40], [SS45], [SS57] and [SS58]. We also find studies such as [SS9], [SS27] and [SS35] where [SS9] uses the open-source dataset "Apache POI", [SS27] uses the Iris dataset, and [SS35] uses AEEEM dataset collected by D'Ambros.

4.3. RQ3: Datasets

We report NASA dataset as a widely used dataset for software fault prediction is NASA dataset. We divide datasets into three categories: Gold, Silver and Bronze standard datasets. Gold standard Datasets include datasets which are from highly ranked repositories such as UCI repository, NASA MDP, GIT or PROMISE. Studies that use Gold standard datasets include [SS2], [SS3], [SS4], [SS5], [SS8], [SS10], [SS15], [SS14], [SS18], [SS20], [SS23], [SS30], [SS34], [SS36], [SS38], [SS39], [SS41], [SS50], [SS53], [SS55], [SS57], [SS59], [SS62], [SS64] and [SS65]. Silver Standard Datasets category includes those repositories which are famous but not highly ranked. Selected studies making use of such datasets are [SS12], [SS13], [SS16], [SS24], [SS26], [SS28], [SS29], [SS31], [SS32], [SS35], [SS37], [SS42], [SS44], [SS45], [SS46], [SS54], [SS56] and [SS58]. Lastly, bronze standard datasets are those datasets or repositories which are neither well known such as "Fit datasets" and nor they are used in majority of studies. Studies using such datasets include [SS1], [SS6], [SS7], [SS9], [SS12], [SS17], [SS22], [SS25], [SS40], [SS49], [SS51], [SS52], [SS61], and [SS66]. Steel defect dataset is used in one of the study where [3] use a Turkish White Goods dataset, other datasets such as kaggle, Eclipse and Java datasets are mentioned but not used as sole sources for datasets repositories. Certain studies such as [SS7], [SS20], [SS21] and [SS38] used same model applied on combination of different datasets. There are few studies that use same model on the combination of different datasets. We notice that there are different techniques of DM, ML and DL that are applied using different datasets. We also find several datasets are based on software metrics. We present list of large publicly available datasets in [Table 13](#).

4.3.1. PROMISE

It is the largest dataset repository in the area of software engineering. It includes many datasets from the different areas of software engineering. It provides different categories of datasets for defects. We present metrics used in various datasets available in PROMISE repository are presented in [Table 10](#).

4.3.2. Bug prediction datasets

It presents five open source software systems which includes the source code metrics more precisely in the area of change metrics. On the basis of this dataset the performance of fault prediction methods can be improved.

Table 11
Metrics used in bug prediction dataset.

Metrics name	Known as
Number of other classes that reference the class	FanIn
Number of other classes referenced by the class	FanOut
Number of attributes	NOA
Number of public attributes	NOPA
Number of private attributes	NOPRA
Number of attributes inherited	NOAI
Number of lines of code	LOC
Number of methods	NOM
Number of public methods	NOPM
Number of private methods	NOPM
Number of methods inherited	NOMI

Table 12
Metrics used in GitHub bug dataset.

List of datasets	Systems	LOC	System name
API Documentation	AD	Weighted Methods per Class	WMC
Clone Classes	CC	Total Number of Statements	TNOS
Clone Coverage	CC	Total Number of Setters	TNS
Clone Complexity	CCO	Total Number of Public Methods	TNPM
Clone Instances	CI	Total Number of Public Attributes	TNPA
Clone Line Coverage	CLC	Total Number of Methods	TNM
Clone Logical Line Coverage	CLLC	Total Number of Local Setters	TNLS
Comment Density	CD	Total Number of Local Public Meth	TNLPM
Comment Lines of Code	CLOC	Total Number of Local Public Attr	TNLPA
Coupling Between Object classes	CBO	Total Number of Local Methods	TNLM
Coupling Between Obj. classes Inv	CBOI	Total Number of Local Getters	TNLG
Depth of Inheritance Tree	DIT	Total Number of Local Attributes	TNLA
Documentation Lines of Code	DLOC	Total Number of Getters	TNG
Lack of Cohesion in Methods 5	LCOM5	Total Number of Attributes	TNA
Lines of Code	LOC	Total Logical Lines of Code	TLLOC
Lines of Duplicated Code	LDC	Total Lines of Cod	TLLOC
Logical Lines of Code	LLOC	Total Comment Lines of Code	TCLOC
Logical Lines of Duplicated Code	LLDC	Total Comment Density	TCD
Nesting Level	NL	Response set For Class	RFC
Nesting Level Else-If	NLE	Public Undocumented API	PUA
Number of Ancestors	NOA	Public Documented API	PDA
Number of Attributes	NA	Number of Statements	NOS
Number of Children	NOC	Number of Setters	NS
Number of Descendants	NOD	Number of Public Methods	NPM
Number of Getters	NG	Number of Public Attributes	NPA
Number of Incoming Invocations	NII	Number of Parents	NOP
Number of Local Attributes	NLA	Number of Outgoing Invocations	NOI
Number of Local Public Attributes	NLPA	Number of Local Public Methods	NLPM
Number of Local Methods	NLM	Number of Local Setters	NLS
Number of Local Getters	NLG	Number of Methods	NM

4.3.3. GitHub bug dataset

The formation of this dataset is based on 15 Java systems from GitHub and constructed a fault localization dataset at class-level and file-level. In this dataset, our area of interest is class-level metrics. In one of the selected primary studies, this dataset was applied to 13 different ML(machine learning) algorithms to investigate which algorithm family performs the best in fault prediction. This dataset includes many source code metrics in the dataset and used these measurements as independent variables in the machine learning process. We present metrics used in GitHub bug datasets in [Table 12](#).

4.3.4. Unified bug dataset

In this dataset, we find all classes (and files) merged into one large dataset which consists of 47,618 elements (43,744 for files). The unified bug dataset contains class-level metrics of Open Static Analyzer [94] and Moose.

4.3.5. Statistics of dataset

We present datasets and their details, models and systems in which we report their usage in [Tables 12](#) and [13](#). We present their specific details such lines of code in each dataset used to extract the source code metrics at class level in [Table 13](#).

Table 13
List of selected datasets.

List of datasets	Systems	LOC	System Name
Github Bud Dataset	14	1,707,446	Antlr 4, Broadleaf Commerce, Ceylon IDE Eclipse Plugin, Elasticsearch, Hazelcast, JUnit, MapDB, mcMMO, MCT, Neo4J, Netty, OrientDB, Titan
PROMISE	14	2,805,253	Ant, Camel, Ckjm, Forrest, Ivy, JEdit, Log4J, Lucene, PBeans, Poi, Synapse, Velocity, Xalan, Xerces
Bug Prediction Dataset	5	1,171,220	Eclipse JDT Core, Eclipse PDE UI, Equinox Framework, Lucene, Mylyn

4.4. RQ3: Metrics

We find datasets used in [SS7], [SS8], [SS20], [SS22], [SS25], [SS26], [SS32], [SS39], [SS42] and [SS54] are based on CK metrics. We also find studies such as [SS2], [SS3], [SS11], [SS15], [SS18], [SS29], [SS31], [SS41], [SS58], [SS64] and [SS65] making use of LOC metrics. Halstead Metrics is also a commonly used metrics and we find its used in selected studies including [SS2], [SS5], [SS11], [SS12], [SS16], [SS18], [SS24], [SS29], [SS31], [SS38], [SS41], [SS50], [SS64] and [SS65]. We also find use of McCabe Metrics in [SS2], [SS5], [SS12], [SS15], [SS16], [SS18], [SS24], [SS29], [SS31], [SS38], [SS41], [SS50], [SS55], [SS64] and [SS65]. We identify few studies use other metrics like QMOOD metrics in their studies. [SS6], [SS16], [SS23], [SS25], [SS40], [SS43], [SS44], [SS45], [SS49], [SS61], [PS65] use other models for software fault prediction.

We observe that most Common datasets used for software fault prediction are NASA and PROMISE datasets. We also find better results in the case of NASA and PROMISE datasets as compared to data sets in repositories such as Kaggle, GIT, Eclipse, Java datasets, Turkish White Goods, and Fit datasets. We also report that various metrics used in different studies are McCabe, Halstead, LOC, CK, QMOOD etc., where CK metrics is the most widely used metrics and it perform better as compared to other metrics for results generation.

We find metrics and associated datasets for SFP considering method-level, class-level or module-level for their definition. We find method-level metrics as more dominant as compared to process metrics and traditional source code metrics. However, process-level metrics have more potential but they are used in a few studies. We observe that about 59 percent of selected studies use method-level metrics and the rest either use class level or module-level metrics. We analyze and highlight that NASA and PROMISE datasets provide method-level metrics with a lesser number of datasets containing class-level or module level information. Our analysis also shows that the software systems against datasets are prepared were written using C/C++ or Java languages. We group calculated metrics into 6 categories, which are the following:

Cohesion metrics: It measure to what extent the source code elements are coherent in the system.

Complexity metrics: It measure the complexity of source code elements (typically algorithms).

Coupling metrics: It measure the amount of inter-dependencies of source code elements.

Documentation metrics: It measure the amount of comments and documentation of source code elements in the system.

Inheritance metrics: It measure the different aspects of the inheritance hierarchy of the system.

Size metrics: It measure the basic properties of the analyzed system in terms of different carnality (e.g. number of code lines, number of classes or methods).

We find studies such as [95,96] using source code metrics to predict the faults especially cohesion metrics, complexity metrics, coupling metrics, documentation metrics, inheritance metrics and size metrics. Public datasets based on these metrics are available publicly like PROMISE [97], Eclipse Bug Dataset [98], Bug Prediction Dataset [99] and GitHub Bug Dataset [100]. We find SFP using source code metrics to predict faulty classes needs more usage with deep learning models. We find fault prediction using metrics such as CK, MOOD and Martin, where bug dataset available in PROMISE [97], Bug Prediction Dataset [99], GitHub Bug Dataset [100] and Unified Bug Dataset [94] are used less frequently. It is worth highlighting that datasets must be labeled and faults should be associated with the relevant classes. These datasets should be publicly available with availability of source code. PROMISE [97], item Bug Prediction Dataset [100], GitHub Bug Dataset [99] and Unified Bug Dataset [94] fulfill all these requirements. Metrics used in PROMISE are presented in Table 10 and metrics used in Bug Prediction Datasets are presented in Table 11.

4.5. RQ4: Performance evaluation and tools usage

We report the use of various performance measures that include precision, accuracy, recall, specificity, precision, FPR, FNR, AUC, and F-measure for data mining, and machine learning models performance comparison. However, deep learning studies use accuracy, TNR, and FNR as their performance evaluation. H-measure, G-mean and mean square error are used rarely as performance

Table 14
Performance measured used in studies.

Performance metric	Studies
Accuracy	[SS1], [SS7], [SS8], [SS9], [SS14], [SS15], [SS24], [SS26], [SS34], [SS37], [SS38], [SS41], [SS50], [SS53], [SS58], [SS63], [SS64], [SS65]
Recall (sensitivity, True positive rate TPR)	[SS1], [SS5], [SS7], [SS8], [SS30], [SS41], [SS55], [SS63], [SS64], [SS65], [SS66], [SS67]
Specificity (TNR)	[SS1], [SS7], [SS55], [SS62], [SS63], [SS65], [SS66], [SS67]
Precision/Correctness	[SS7], [SS8], [SS22], [SS25], [SS30], [SS31], [SS37], [SS38], [SS41], [SS44], [SS45], [SS55], [SS53], [SS63], [SS65], [SS66], [SS67]
False Positive rate (FPR)	[SS1], [SS5], [SS10], [SS41], [SS58], [SS65], [SS66], [SS67]
False Negative rate (FNR)	[SS1], [SS5], [SS40], [SS65], [SS66], [SS67]
AUC (Area Under Curve)	[SS1], [SS4], [SS7], [SS16], [SS23], [SS24], [SS26], [SS40], [SS46], [SS63]
ROC	[SS1], [SS7], [SS8], [SS23], [SS46], [SS53]
F measure	SS3], [SS25], [SS29], [SS31], [SS37], [SS37], [SS43], [SS44], [SS45], [SS63], [SS66], [SS67]
Mean Square error	[SS2], [SS12], [SS17], [SS52], [SS65]

measures. We analyze how authors of selected primary studies do performance evaluation of their models. We provide their details using Table 14 and we report that accuracy, recall, and F-measure are commonly used for performance evaluation.

We did not find any study in which Mean Absolute Error(MAE) is used. We find methods for data preparation, classification, regression, clustering, association rules mining, and visualization in our chosen research. Various studies use WEKA¹ that include [SS1], [SS3], [SS12], [SS35] and [SS41]. MATLAB² is a well-known tool used for software fault prediction we find its use in [SS15], [SS23], [SS27], [SS30], [SS53] and [PS65]. Few studies, such as [SS20] used R studio for software fault prediction. We find the use of various development environments for Python used in [SS7], [SS62] and [SS64]. Our analysis shows the use of Python IDE for software fault prediction is more evident than tools such as WEKA or R Studio.

4.6. RQ5: Models for SFP

We separate the selected studies based on identified constructs. we observe that the most widely used constructs are software fault, software fault prediction and software defect prediction. There are studies that make use of deep learning or machine learning techniques with a minor percentage using deep learning. We believe this is due to two reasons. Firstly, most datasets have small to average size and we require more data for deep learning. Secondly, deep learning techniques are a recent phenomenon. Mostly studies use different tools for their experiments. Constructs such as dependent and independent variables and software fault detection are used less frequently.

Machine learning techniques are the most widely used as compared to data mining or deep learning techniques in the field of software fault prediction. We observed that classification techniques within machine learning techniques are used for software fault prediction in more cases. Researchers using data mining techniques for SFP focus on classification techniques to predict faulty and non-faulty modules.

We find deep learning methods are effective when researchers consider larger datasets. We observe that there is a requirement to do comparative analysis between use of data mining, machine learning or deep learning techniques for SFP. Our analysis shows that support vector machine and decision tree outperform rest of the data mining techniques used for SFP. This is because we find SVM reporting better results in case of machine learning or data mining models for binary classification problems.

In deep learning, Convolutional Neural Network performance is better as compared to Multilayer perceptrons but CNN consume more. However there are many deep learning techniques such as Radial Basis Function Network (RBFN), Autoencoders, Bidirectional Long short-term memory (BiLSTM) which can give better results in the classification of faulty and non-faulty modules consuming less time. If we compare the overall performance of data mining, machine learning and deep learning algorithms found in studies, deep learning algorithms perform better than data mining or machine learning algorithms. We also conclude that different kinds of feature selection algorithms used in primary studies could not have a considerable effect on the performance of the algorithms in case of data mining, machine learning or deep learning.

When we are using deep learning techniques on large datasets, different hyper-parameter tuning can be used for precise results. Comparison between the metrics should be conducted to evaluate the performance measure. We report that RBFN and stacking is not used and we find very few studies using K-fold cross validation. Feature selection is an important topic and wrapper-based feature selection may provide a new perspective on filter-based approaches for the software engineering community. We also report use of feature reduction techniques such as Rough Set Theory or feature selection technique such as Principal component analysis

¹ <https://weka.en.softonic.com/>.

² <https://matlab.en.softonic.com/>.

may benefit SFP. We also find lack of cost benefit analysis of models and there are very few studies which use of code-smells for SFP. We also find out performance of different classifiers such as genetic programming to build a computer model that can predict faults based on a selected metrics.

5. Conclusion

We performed a systematic literature review to analyze the performance of data mining, machine learning, and deep learning techniques for software fault prediction. We selected 68 primary studies and summarized characteristics of selected studies based on datasets, techniques and performance measures. We assessed the performance of models classifying them into data mining, machine learning, and deep learning techniques for software fault prediction. We found out which of the techniques are preferred by researchers in each category i.e., data mining, machine learning, and deep learning. We also compared performance reported in each of the selected primary studies and we reported which of the performance measure is used in each of these studies.

We report that data mining and machine learning are widely used for software fault prediction. However, researchers used ensemble techniques and deep learning methods less frequently for software fault prediction. There is need to work on using same techniques on combinations of different datasets and there is a requirement of more and larger datasets in public domain.

CRedit authorship contribution statement

Iqra Batool: Conceptualization, Methodology, Data curation, Writing – original draft, Writing – review & editing. **Tamim Ahmed Khan:** Visualization, Investigation, Supervision, Writing – review & editing.

Declaration of competing interest

No author associated with this paper has disclosed any potential or pertinent conflicts which may be perceived to have impending conflict with this work. For full disclosure statements refer to <https://doi.org/10.1016/j.compeleceng.2022.107886>.

References

- [1] Hassan F, Farhan S, Fahiem MA, Tauseef H. A review on machine learning techniques for software defect prediction. *Tech J* 2018;23(02):63–71.
- [2] Turabieh H, Mafarja M, Li X. Iterated feature selection algorithms with layered recurrent neural network for software fault prediction. *Expert Syst Appl* 2019;122:27–42.
- [3] Malhotra R. A systematic review of machine learning techniques for software fault prediction. *Appl Soft Comput* 2015;27:504–18.
- [4] Association I. Computer systems and software engineering: concepts, methodologies, tools, and applications. 2017, p. 1–2100. <http://dx.doi.org/10.4018/978-1-5225-3923-0>.
- [5] Radjenović D, Herić M, Torkar R, Živković Aš. Software fault prediction metrics: A systematic literature review. *Inf Softw Technol* 2013;55(8):1397–418.
- [6] Jakhar AK, Rajnish K. Software fault prediction with data mining techniques by using feature selection based models. *Int J Electr Eng Inf* 2018;10(3).
- [7] Manjula C, Florence L. Deep neural network based hybrid approach for software defect prediction using software metrics. *Cluster Comput* 2019;22(4):9847–63.
- [8] Catal C, Diri B. A systematic review of software fault prediction studies. *Expert Syst Appl* 2009;36(4):7346–54.
- [9] Kanmani S, Uthariaraj VR, Sankaranarayanan V, Thambidurai P. Object-oriented software fault prediction using neural networks. *Inf Softw Technol* 2007;49(5):483–92.
- [10] Hall T, Beecham S, Bowes D, Gray D, Counsell S. A systematic literature review on fault prediction performance in software engineering. *IEEE Trans Softw Eng* 2011;38(6):1276–304.
- [11] Choudhary GR, Kumar S, Kumar K, Mishra A, Catal C. Empirical analysis of change metrics for software fault prediction. *Comput Electr Eng* 2018;67:15–24.
- [12] Sandhu PS, Singh V, Gupta V, Kaur M, Manhas S, Sidhu R. A k-means based clustering approach for finding faulty modules in open source software systems. *World Acad Sci Eng Technol* 2010;72:654–8.
- [13] Arora I, Saha A. Software fault prediction using firefly algorithm. *Int J Intell Eng Inf* 2018;6(3–4):356–77.
- [14] Yohannese CW, Li T, Simfukwe M, Khurshid F. Ensembles based combined learning for improved software fault prediction: A comparative study. In: 2017 12th international conference on intelligent systems and knowledge engineering. IEEE; 2017, p. 1–6.
- [15] Rathore SS, Kumar S. An empirical study of ensemble techniques for software fault prediction. *Appl Intell* 2021;51(6):3615–44.
- [16] Liu W, Liu S, Gu Q, Chen J, Chen X, Chen D. Empirical studies of a two-stage data preprocessing approach for software fault prediction. *IEEE Trans Reliab* 2015;65(1):38–53.
- [17] Malhotra R, Khan K. A study on software defect prediction using feature extraction techniques. In: 2020 8th international conference on reliability, infocom technologies and optimization (Trends and Future Directions). IEEE; 2020, p. 1139–44.
- [18] Durelli VH, Durelli RS, Borges SS, Endo AT, Eler MM, Dias DR, Guimarães MP. Machine learning applied to software testing: A systematic mapping study. *IEEE Trans Reliab* 2019;68(3):1189–212.
- [19] Cao H. A systematic study for learning-based software defect prediction. *J Phys Conf Ser* 2020;1487(1):012017.
- [20] Omri S, Sinz C. Deep learning for software defect prediction: A survey. In: Proceedings of the IEEE/ACM 42nd international conference on software engineering workshops. 2020, p. 209–14.
- [21] Kitchenham B, Brereton OP, Budgen D, Turner M, Bailey J, Linkman S. Systematic literature reviews in software engineering—a systematic literature review. *Inf Softw Technol* 2009;51(1):7–15.
- [22] Chen Y, Shen X-h, Du P, Ge B. Research on software defect prediction based on data mining. In: 2010 the 2nd international conference on computer and automation engineering, vol. 1. IEEE; 2010, p. 563–7.
- [23] Afzal W, Torkar R, Feldt R. A systematic review of search-based testing for non-functional system properties. *Inf Softw Technol* 2009;51(6):957–76.
- [24] Gao K, Khoshgoftaar TM, Wang H, Seliya N. Choosing software metrics for defect prediction: an investigation on feature selection techniques. *Softw - Pract Exp* 2011;41(5):579–606.
- [25] Zheng J. Cost-sensitive boosting neural networks for software defect prediction. *Expert Syst Appl* 2010;37(6):4537–43.
- [26] Li M, Zhang H, Wu R, Zhou Z-H. Sample-based software defect prediction with active and semi-supervised learning. *Autom Softw Eng* 2012;19(2):201–30.

- [27] Sun Z, Song Q, Zhu X. Using coding-based ensemble learning to improve software defect prediction. *IEEE Trans Syst Man Cybern Part C (Applications and Reviews)* 2012;42(6):1806–17.
- [28] Arar OF, Ayan K. Software defect prediction using cost-sensitive neural network. *Appl Soft Comput* 2015;33:263–77.
- [29] Wong WE, Debroy V, Golden R, Xu X, Thuraisingham B. Effective software fault localization using an RBF neural network. *IEEE Trans Reliab* 2011;61(1):149–69.
- [30] Malhotra R, Jain A. Fault prediction using statistical and machine learning methods for improving software quality. *J Inf Process Syst* 2012;8(2):241–62.
- [31] Czibula G, Marian Z, Czibula IG. Software defect prediction using relational association rule mining. *Inform Sci* 2014;264:260–78. <http://dx.doi.org/10.1016/j.ins.2013.12.031>.
- [32] Lv F, Wen C, Bao Z, Liu M. Fault diagnosis based on deep learning. In: 2016 American control conference. IEEE; 2016, p. 6851–6.
- [33] Cai X, Niu Y, Geng S, Zhang J, Cui Z, Li J, Chen J. An under-sampled software defect prediction method based on hybrid multi-objective cuckoo search. *Concurr Comput: Pract Exper* 2020;32(5):e5478.
- [34] Turhan B, Misirlı AT, Bener A. Empirical evaluation of the effects of mixed project data on learning defect predictors. *Inf Softw Technol* 2013;55(6):1101–18.
- [35] Catal C, Sevim U, Diri B. Practical development of an Eclipse-based software fault prediction tool using Naive Bayes algorithm. *Expert Syst Appl* 2011;38(3):2347–53.
- [36] Moeyersoms J, de Fortuny EJ, Dejaeger K, Baesens B, Martens D. Comprehensible software fault and effort prediction: A data mining approach. *J Syst Softw* 2015;100:80–90.
- [37] Jin C, Jin S-W. Prediction approach of software fault-proneness based on hybrid artificial neural network and quantum particle swarm optimization. *Appl Soft Comput* 2015;35:717–25.
- [38] Deng X, Yuan R, Xiao Z, Li T, Wang KLL. Fault location in loop distribution network using SVM technology. *Int J Electr Power Energy Syst* 2015;65:254–61.
- [39] Singh P, Pal NR, Verma S, Vyas OP. Fuzzy rule-based approach for software fault prediction. *IEEE Trans Syst Man Cybern Syst* 2016;47(5):826–37.
- [40] Wang S, Liu T, Nam J, Tan L. Deep semantic feature learning for software defect prediction. *IEEE Trans Softw Eng* 2018;46(12):1267–93.
- [41] Singh Y, Kaur A, Malhotra R. Empirical validation of object-oriented metrics for predicting fault proneness models. *Softw Qual J* 2010;18(1):3–35.
- [42] Malhotra R, Singh Y. On the applicability of machine learning techniques for object oriented software fault prediction. *Softw Eng Int J* 2011;1(1):24–37.
- [43] Huda S, Alyahya S, Ali MM, Ahmad S, Abawajy J, Al-Dossari H, Yearwood J. A framework for software defect prediction and metric selection. *IEEE Access* 2017;6:2844–58.
- [44] Abebe SL, Arnaoudova V, Tonella P, Antoniol G, Gueheneuc Y-G. Can lexicon bad smells improve fault prediction? In: 2012 19th working conference on reverse engineering. IEEE; 2012, p. 235–44.
- [45] Malhotra R, Bansal AJ. Fault prediction considering threshold effects of object-oriented metrics. *Expert Syst* 2015;32(2):203–19.
- [46] Zheng W, Hu D, Wang J. Fault localization analysis based on deep neural network. *Math Probl Eng* 2016;2016.
- [47] Singh PD, Chug A. Software defect prediction analysis using machine learning algorithms. In: 2017 7th International conference on cloud computing, data science & engineering-confluence. IEEE; 2017, p. 775–81.
- [48] Ren J, Qin K, Ma Y, Luo G. On software defect prediction using machine learning. *J Appl Math* 2014;2014.
- [49] Jayanthi R, Florence L. Software defect prediction techniques using metrics based on neural network classifier. *Cluster Comput* 2019;22(1):77–88.
- [50] Jin C, Jin S-W, Ye J-M. Artificial neural network-based metric selection for software fault-prone prediction model. *IET Softw* 2012;6(6):479–87.
- [51] Fan G, Diao X, Yu H, Yang K, Chen L. Software defect prediction via attention-based recurrent neural network. *Sci Program* 2019;2019.
- [52] Chen J, Liu S, Liu W, Chen X, Gu Q, Chen D. A two-stage data preprocessing approach for software fault prediction. In: 2014 eighth international conference on software security and reliability. IEEE; 2014, p. 20–9.
- [53] Chen L, Babar MA, Zhang H. Towards an evidence-based understanding of electronic data sources. In: 14th international conference on evaluation and assessment in software engineering. 2010, p. 1–4.
- [54] Hall T, Bowes D. The state of machine learning methodology in software fault prediction. In: 2012 11th international conference on machine learning and applications, vol. 2. IEEE; 2012, p. 308–13.
- [55] Rong X, Li F, Cui Z. A model for software defect prediction using support vector machine based on CBA. *Int J Intell Syst Technol Appl* 2016;15(1):19–34.
- [56] Akour M, Alsmadi I, Alazzam I. Software fault proneness prediction: a comparative study between bagging, boosting, and stacking ensemble and base learner methods. *Int J Data Anal Tech Strateg* 2017;9(1):1–16. <http://dx.doi.org/10.1504/IJDATS.2017.10003991>.
- [57] Singh A, Bhatia R, Singhrova A. Taxonomy of machine learning algorithms in software fault prediction using object oriented metrics. *Procedia Comput Sci* 2018;132:993–1001.
- [58] Zhou T, Sun X, Xia X, Li B, Chen X. Improving defect prediction with deep forest. *Inf Softw Technol* 2019;114:204–16.
- [59] Abdi Y, Parsa S, Seyfari Y. A hybrid one-class rule learning approach based on swarm intelligence for software fault prediction. *Innov Syst Softw Eng* 2015;11(4):289–301.
- [60] Rathore SS, Gupta A. Validating the effectiveness of object-oriented metrics over multiple releases for predicting fault proneness. In: 2012 19th Asia-Pacific software engineering conference, vol. 1. IEEE; 2012, p. 350–5.
- [61] Phan AV, Le Nguyen M. Convolutional neural networks on assembly code for predicting software defects. In: 2017 21st Asia Pacific symposium on intelligent and evolutionary systems. IEEE; 2017, p. 37–42.
- [62] Arshad A, Riaz S, Jiao L, Murthy A. Semi-supervised deep fuzzy c-mean clustering for software fault prediction. *IEEE Access* 2018;6:25675–85.
- [63] Cao Q, Sun Q, Cao Q, Tan H. Software defect prediction via transfer learning based neural network. In: 2015 First international conference on reliability systems engineering. IEEE; 2015, p. 1–10.
- [64] Shanthini A, Chandrasekaran R. Analyzing the effect of bagged ensemble approach for software fault prediction in class level and package level metrics. In: International conference on information communication and embedded systems. IEEE; 2014, p. 1–5.
- [65] Sharma D, Chandra P. Software fault prediction using machine-learning techniques. In: Smart computing and informatics. Springer; 2018, p. 541–9.
- [66] Rana R, Staron M, Berger C, Hansson J, Nilsson M, Meding W. The adoption of machine learning techniques for software defect prediction: An initial industrial validation. In: Kravets AG, Shcherbakov M, Kultsova MV, Iijima T, editors. Knowledge-based software engineering - 11th joint conference. Communications in computer and information science, vol. 466, Springer; 2014, p. 270–85. http://dx.doi.org/10.1007/978-3-319-11854-3_23.
- [67] Coelho RA, dos RN Guimarães F, Esmín AA. Applying swarm ensemble clustering technique for fault prediction using software metrics. In: 2014 13th international conference on machine learning and applications. IEEE; 2014, p. 356–61.
- [68] Sethi T, Gagandeep. Improved approach for software defect prediction using artificial neural networks. In: 2016 5th international conference on reliability, infocom technologies and optimization (Trends and Future Directions). 2016, p. 480–5. <http://dx.doi.org/10.1109/ICRITO.2016.7785003>.
- [69] Chatterjee S, Nigam S, Roy A. Software fault prediction using neuro-fuzzy network and evolutionary learning approach. *Neural Comput Appl* 2017;28(1):1221–31.
- [70] Kaur D, Kaur A, Gulati S, Aggarwal M. A clustering algorithm for software fault prediction. In: 2010 international conference on computer and communication technology. IEEE; 2010, p. 603–7.
- [71] Wu F. Empirical validation of object-oriented metrics on NASA for fault prediction. In: Advances in information technology and education. Springer; 2011, p. 168–75.
- [72] Bejjanki KK, Gyani J, Gugulothu N. Class imbalance reduction (CIR): a novel approach to software defect prediction in the presence of class imbalance. *Symmetry* 2020;12(3):407.

- [73] Khuat TT, Le MH. Ensemble learning for software fault prediction problem with imbalanced data. *Int J Electr Comput Eng* 2019;9(4):3241.
- [74] Chen L, Fang B, Shang Z. Software fault prediction based on one-class SVM. In: 2016 international conference on machine learning and cybernetics, vol. 2. IEEE; 2016, p. 1003–8.
- [75] Shepperd M, Bowes D, Hall T. Researcher bias: The use of machine learning in software defect prediction. *IEEE Trans Softw Eng* 2014;40(6):603–16.
- [76] Wei H, Shan C, Hu C, Sun H, Lei M. Software defect distribution prediction model based on NPE-SVM. *China Commun* 2018;15(5):173–82.
- [77] Mundada D, Murade A, Vaidya O, Swathi J. Software fault prediction using artificial neural network and resilient back propagation. *Int J Comput Sci Eng* 2016;5(03).
- [78] Zhang P, Chang Y-t. Software fault prediction based on grey neural network. In: 2012 8th international conference on natural computation. IEEE; 2012, p. 466–9.
- [79] Al Qasem O, Akour M. Software fault prediction using deep learning algorithms. *Int J Open Source Softw Process (IJOSSP)* 2019;10(4):1–19.
- [80] Rhmann W, Ansari GA. Ensemble techniques-based software fault prediction in an open-source project. In: Research anthology on usage and development of open source software. IGI Global; 2021, p. 693–709.
- [81] Al Qasem O, Akour M, Alenezi M. The influence of deep learning algorithms factors in software fault prediction. *IEEE Access* 2020;8:63945–60.
- [82] Chen J, Hu K, Yu Y, Chen Z, Xuan Q, Liu Y, Filkov V. Software visualization and deep transfer learning for effective software defect prediction. In: Rothmel G, Bae D, editors. ICSE '20: 42nd international conference on software engineering. ACM; 2020, p. 578–89. <http://dx.doi.org/10.1145/3377811.3380389>.
- [83] Chen J, Hu K, Yang Y, Liu Y, Xuan Q. Collective transfer learning for defect prediction. *Neurocomputing* 2020;416:103–16.
- [84] ÖZTÜRK E, Birant KU, Birant D. An ordinal classification approach for software bug prediction. *Dokuz Eylül Üniversitesi Mühendislik Fakültesi Fen Ve Mühendislik Dergisi* 2019;21(62):533–44.
- [85] Balaran A, Vasundra S. Prediction of software fault-prone classes using ensemble random forest with adaptive synthetic sampling algorithm. *Autom Softw Eng* 2022;29(1):1–21.
- [86] Goodfellow I, Bengio Y, Courville A. Deep learning. MIT Press; 2016.
- [87] Abaei G, Selamat A, Fujita H. An empirical study based on semi-supervised hybrid self-organizing map for software fault prediction. *Knowl-Based Syst* 2015;74:28–39.
- [88] Alsaeedi A, Khan MZ. Software defect prediction using supervised machine learning and ensemble techniques: a comparative study. *J Softw Eng Appl* 2019;12(5):85–100.
- [89] Souri A, Hosseini R. A state-of-the-art survey of malware detection approaches using data mining techniques. *Human-Centric Comput Inf Sci* 2018;8(1):1–22.
- [90] Siregar SP, Wanto A. Analysis of artificial neural network accuracy using backpropagation algorithm in predicting process (forecasting). *IJISTECH (International Journal of Information System & Technology)* 2017;1(1):34–42.
- [91] Mathew A, Amudha P, Sivakumari S. Deep learning techniques: An overview. In: International conference on advanced machine learning technologies and applications. Springer; 2020, p. 599–608.
- [92] Li Y, Wang S, Nguyen TN. Fault localization with code coverage representation learning. In: 2021 IEEE/ACM 43rd international conference on software engineering. IEEE; 2021, p. 661–73.
- [93] Chao MA, Kulkarni C, Goebel K, Fink O. Hybrid deep fault detection and isolation: Combining deep neural networks and system performance models. 2019, arXiv preprint [arXiv:1908.01529](https://arxiv.org/abs/1908.01529).
- [94] Ferenc R, Tóth Z, Ladányi G, Siket I, Gyimóthy T. A public unified bug dataset for java. In: Proceedings of the 14th international conference on predictive models and data analytics in software engineering. 2018, p. 12–21.
- [95] Kumar L, Sureka A. Analyzing fault prediction usefulness from cost perspective using source code metrics. In: 2017 Tenth international conference on contemporary computing. IEEE; 2017, p. 1–7.
- [96] Kumar L, Misra S, Rath SK. An empirical analysis of the effectiveness of software metrics and fault prediction model for identifying faulty classes. *Comput Stand Interfaces* 2017;53:1–32.
- [97] Boetticher G. The PROMISE repository of empirical software engineering data. 2007, <http://promisedata.org/repository>.
- [98] Zimmermann T, Premraj R, Zeller A. Predicting defects for eclipse. In: Third international workshop on predictor models in software engineering (PROMISE'07: ICSE workshops 2007). IEEE; 2007, p. 9.
- [99] D'Ambros M, Lanza M, Robbes R. An extensive comparison of bug prediction approaches. In: 2010 7th IEEE working conference on mining software repositories. IEEE; 2010, p. 31–41.
- [100] Tóth Z, Gyimesi P, Ferenc R. A public bug database of github projects and its application in bug prediction. In: International conference on computational science and its applications. Springer; 2016, p. 625–38.

Iqra Batool: She received her BS degree in computer science from Azad Jammu and Kashmir University. She is currently pursuing Ph.D. degree Khalifa University UAE. Her research interests include data mining, machine learning, deep learning and software fault prediction.

Tamim Ahmad Khan: He received B.E. degree in software engineering from Sheffield University, U.K., MS degree in computer engineering from Taxila University, Pakistan, and Ph.D. from Leicester University. His research interests include service-oriented architectures and software quality assurance.